Projeto G2: Organizador de Estudos com IA Integrada

ProjetoG2_261030 · Entrega da aula de 30/10/2026

Desenvolvimento de Dispositivos Móveis · Universidade La Salle

1. Integrantes que colaboraram na aula de hoje (30/10)

Aplicativo: EduMind. Integrantes da equipe (conforme a entrega de 23/10):

Integrante	Matrícula	Colaborou hoje?
Gustavo Peixoto Siriacov	202411977	[ ] sim [ ] não ← preencher
Tales Ivaniski Galina	202510926	[ ] sim [ ] não ← preencher
Lorenzo Ruggini Lindemann	202511046	[ ] sim [ ] não ← preencher
Eduardo Cassel da Costa	202513138	[ ] sim [ ] não ← preencher

ATENÇÃO (apagar após preencher): o código não registra quem participou da aula de 30/10. Marque acima quem colaborou hoje.

2. O que é o model do EduMind

O model é a camada que guarda e lê os dados. No EduMind ele fica na API (apps/api/src), em SQLite, e o app web só conversa com ela por HTTP: nenhuma tela acessa o banco diretamente. Os dados ficam no servidor, ligados à conta do usuário, e por isso aparecem em qualquer dispositivo com o mesmo login.

- **Estrutura (db/schema.sql):** 12 tabelas com chaves estrangeiras, restrições (unique, check) e índices. É aplicada pelo migrador no primeiro uso do banco (pragma user_version).
- **Acesso (db/db.ts):** quatro atalhos de consulta com parâmetros (one, all, run, insert). Nenhum valor é concatenado no SQL, o que evita injeção de SQL.
- **Repositórios e rotas (modules/*):** cada módulo tem as operações da sua tabela. Antes de ler ou alterar, a API confirma que o registro pertence ao usuário logado.
- **Integridade:** apagar uma disciplina apaga em cascata seus tópicos, metas, aulas, trabalhos e sessões de estudo; o log de acessos sobrevive à exclusão da conta (guarda cópia do nome e do e-mail).

3. Diagrama do modelo de dados (DER)

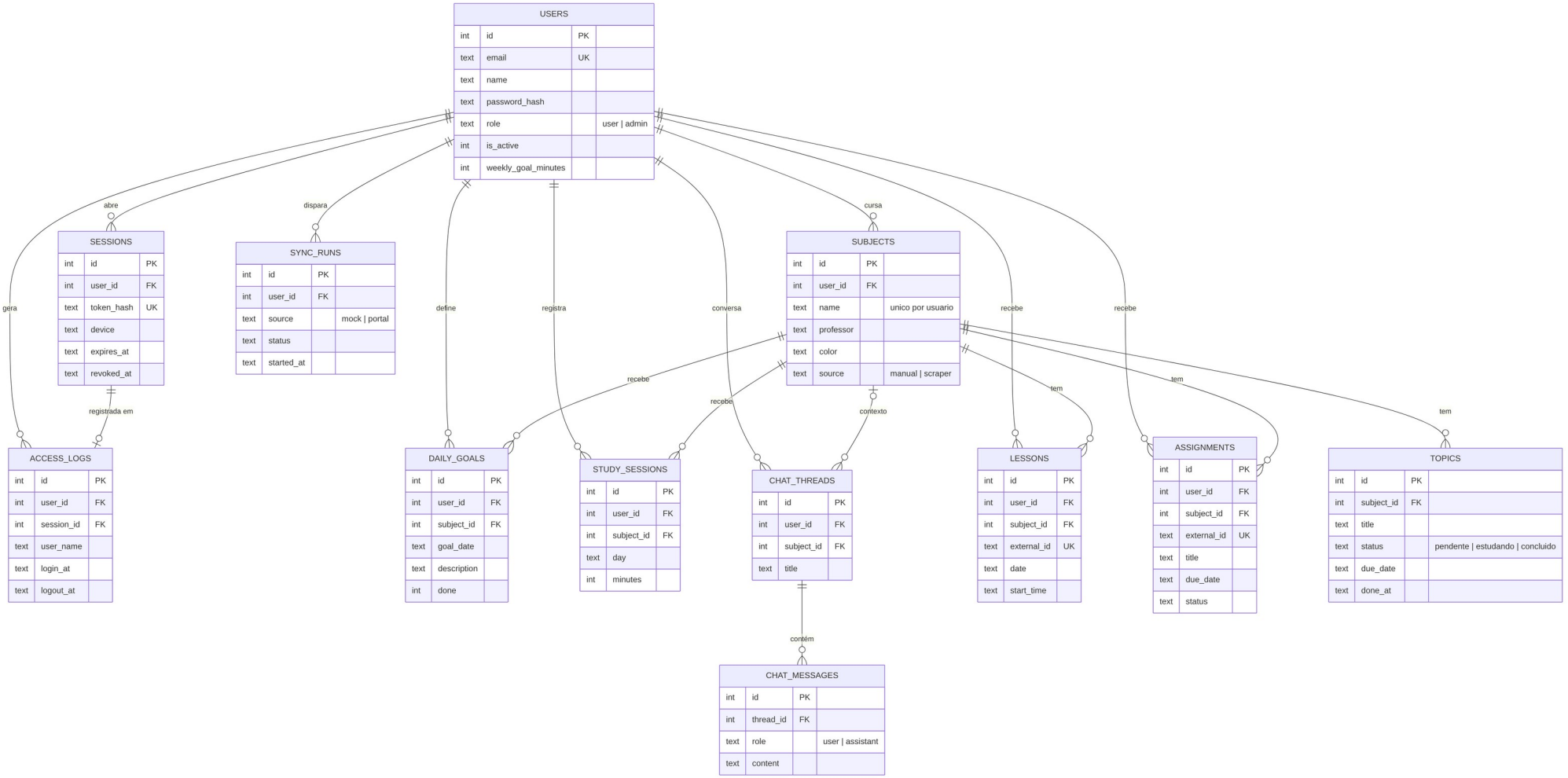


Figura 1 — Tabelas persistidas e seus relacionamentos (cada usuário só enxerga os próprios dados).

Resumo das tabelas:

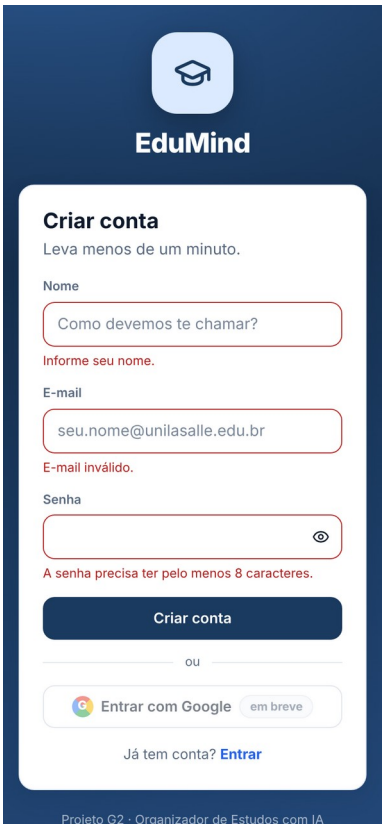
Tabela	O que guarda
users	conta: e-mail, nome, hash da senha (scrypt), papel, ativo/bloqueado, meta semanal
sessions	uma sessão por dispositivo (hash do token, validade, encerramento)
access_logs	log de acesso: nome, data, hora de login e de logout
subjects	disciplinas do usuário (nome único por usuário, professor, sala, cor)
topics	tópicos de cada disciplina com status (pendente, estudando, concluído) e prazo
daily_goals	metas do dia por disciplina, com marcação de concluída
study_sessions	minutos estudados por dia e disciplina (base da tela Progresso)
chat_threads / chat_messages	conversas com o Professor Tutor IA e suas mensagens
lessons / assignments	aulas e trabalhos lidos da faculdade pelo scraper
sync_runs	histórico das sincronizações (status, início, fim, estatísticas)

4. Telas envolvidas na persistência

Cada tela abaixo grava ou lê dados no banco. A tabela relaciona a ação do usuário com a operação feita no model.

Tela	Ação do usuário	Operação no banco
Cadastro	cria a conta	INSERT em users; abre sessão e log (INSERT em sessions e access_logs)
Login	entra no app	SELECT em users; INSERT em sessions e access_logs; UPDATE de users.last_login_at
Disciplinas	cria, edita e exclui disciplina	INSERT / SELECT / UPDATE / DELETE em subjects
Detalhe da disciplina	adiciona tópico e muda o status	INSERT / SELECT / UPDATE / DELETE em topics
Início (metas do dia)	cria e conclui metas	INSERT / SELECT / UPDATE / DELETE em daily_goals
Progresso	registra tempo e vê a semana	INSERT em study_sessions; SELECT com soma por dia e disciplina
Perfil e acessos	edita o perfil e vê os dispositivos	UPDATE em users; SELECT em sessions e access_logs; encerra sessão (UPDATE)
Tutor IA	conversa com o tutor	INSERT em chat_threads e chat_messages; DELETE da conversa
Aulas	sincroniza aulas e trabalhos	INSERT ou UPDATE em lessons, assignments e subjects; registro em sync_runs
Admin	gerencia usuários e vê o log	SELECT em access_logs e sessions; UPDATE e DELETE em users

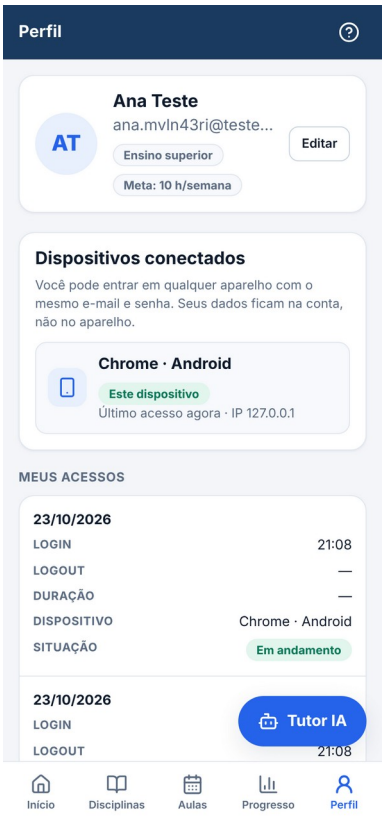
Cadastro e login (users, sessions, access_logs)



Cadastro

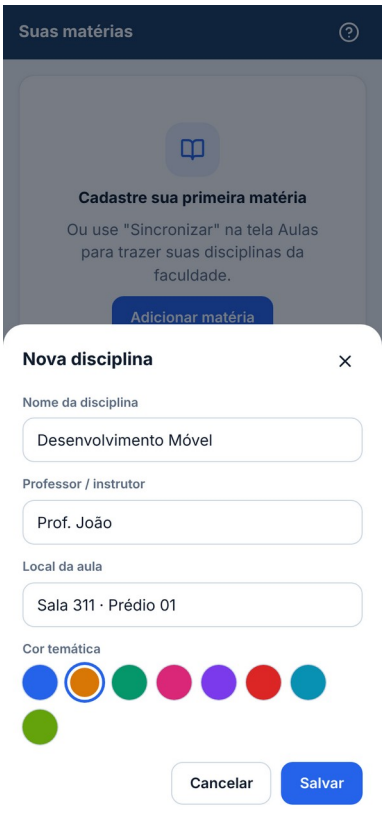


Login

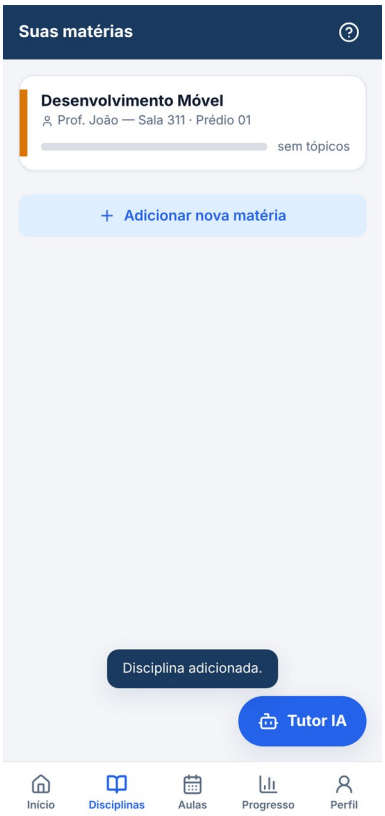


Perfil e acessos

Disciplinas, detalhe e metas (subjects, topics, daily_goals)



Nova disciplina



Disciplinas



Detalhe (tópicos)

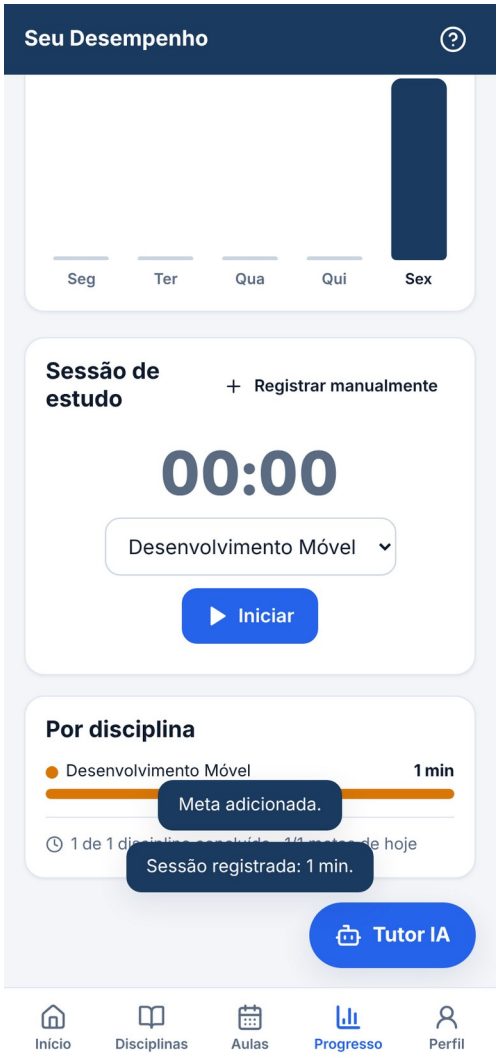


Início vazio

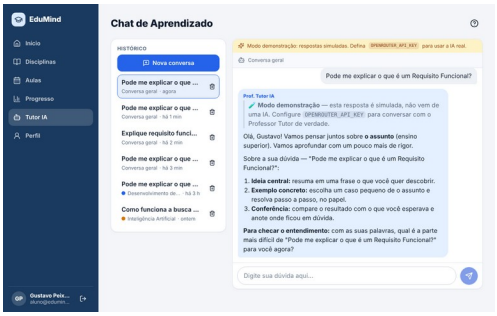


Início com dados (desktop)

Progresso e tutor (study_sessions, chat_*)



Progresso



Tutor IA (desktop)

Aulas sincronizadas (lessons, assignments)

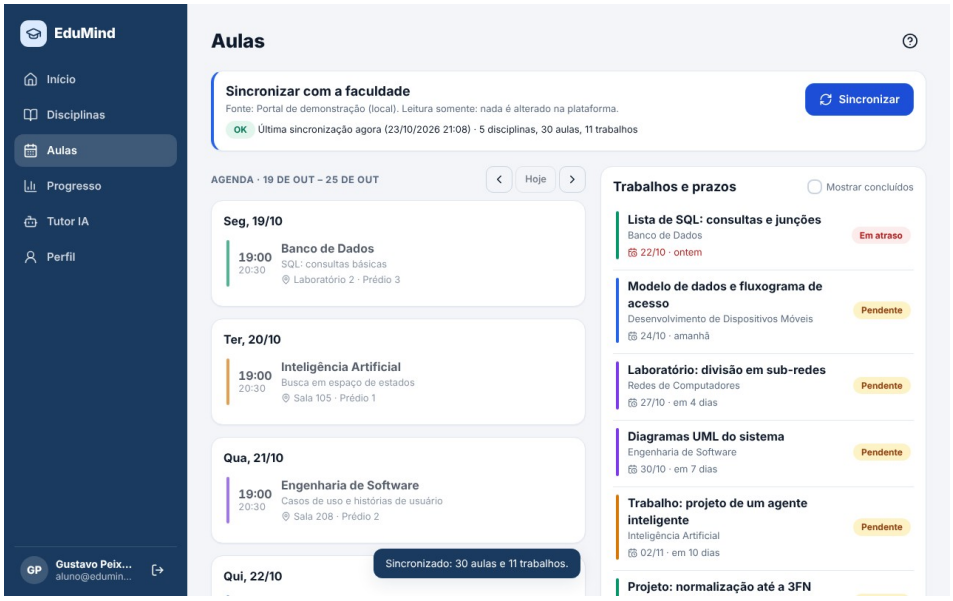
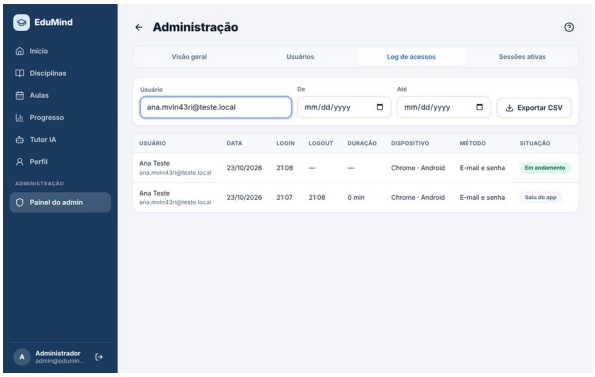
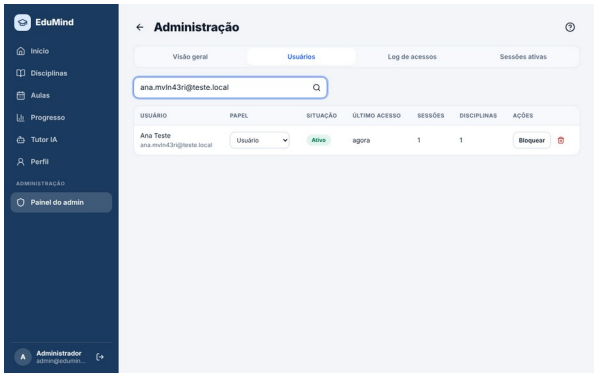


Figura 2 — Aulas e trabalhos importados pelo scraper (desktop).

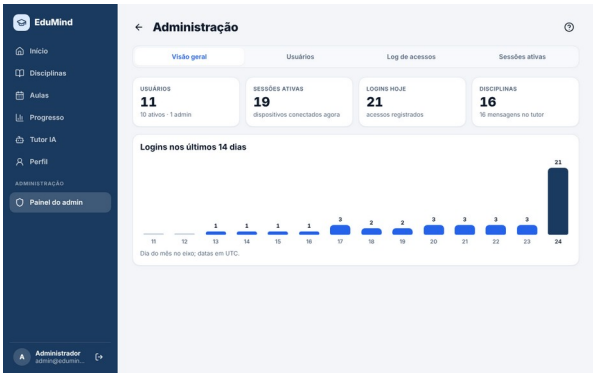
Administração (users, sessions, access_logs)



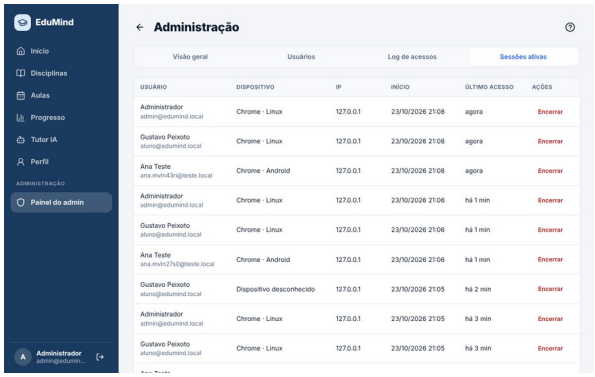
Log de acessos



Usuários



Visão geral



Sessões ativas

5. Código do model: operações com os dados

Trechos reais da API (apps/api/src), reduzidos ao essencial. Todas as consultas usam parâmetros (?).

5.1 Conexão, migração e atalhos de consulta

db/db.ts — migração do esquema

```
export function migrate(db: Db): void {
  const current = db.pragma('user_version', { simple: true }) as number;
  for (let v = current; v < MIGRATIONS.length; v++) {
    const sql = readFileSync(new URL(MIGRATIONS[v]!, import.meta.url), 'utf8');
    db.transaction(() => {
      db.exec(sql);
      db.pragma(`user_version = ${v + 1}`);
    })();
  }
}
```

db/db.ts — atalhos usados por todos os módulos

```
export const one = <T>(db: Db, sql: string, ...params: unknown[]): T | undefined =>
  db.prepare(sql).get(...params) as T | undefined;
export const all = <T>(db: Db, sql: string, ...params: unknown[]): T[] =>
  db.prepare(sql).all(...params) as T[];
export const run = (db: Db, sql: string, ...params: unknown[]) =>
db.prepare(sql).run(...params);
export const insert = (db: Db, sql: string, ...params: unknown[]): number =>
  Number(run(db, sql, ...params).lastInsertRowid);
```

db/schema.sql — exemplo de tabela (restrições e chave estrangeira)

```
create table subjects (
  id          integer primary key autoincrement,
  user_id     integer not null references users(id) on delete cascade,
  name        text    not null collate nocase,      -- único por usuário, sem diferenciar caixa
  professor   text,
  room        text,
  color       text    not null
              check (color glob '#[0-9A-Fa-f][0-9A-Fa-f][0-9A-Fa-f][0-9A-Fa-f][0-9A-Fa-f][0-9A-
Fa-f]'),
  source      text    not null default 'manual' check (source in ('manual','scraper')),
  external_id text,
  created_at  text    not null,
  unique (user_id, name)
);
create unique index subjects_external_idx on subjects (user_id, external_id) where external_id
is not null;
```

5.2 INSERT (create)

modules/auth/users.ts — cria o usuário (tela Cadastro)

```
export function insertUser(db: Db, u: NewUser): UserRow {
  const id = insert(
    db,
    `insert into users (email, name, password_hash, level, role, is_active, created_at)
     values (?, ?, ?, ?, ?, ?, ?)`,
    u.email, u.name, u.passwordHash, u.level ?? 'superior', u.role ?? 'user', u.isActive ===
false ? 0 : 1, u.createdAt,
  );
  return findUserById(db, id)!;
}
```

modules/subjects/routes.ts — cria a disciplina (tela Disciplinas)

```
const id = insert(
  db,
  `insert into subjects (user_id, name, professor, room, color, source, created_at)
   values (?, ?, ?, ?, ?, 'manual', ?)`,
  userId, input.name, input.professor, input.room, input.color, ctx.now(),
);
```

modules/topics/routes.ts — cria o tópico na próxima posição

```
const next = one<{ p: number }>(
  db,
  'select coalesce(max(position), -1) + 1 p from topics where subject_id = ?',
  subject.id,
)!.p;
const id = insert(
  db,
  'insert into topics (subject_id, title, notes, position, due_date) values (?, ?, ?, ?, ?)',
  subject.id, input.title, input.notes, next, input.dueDate,
);
```

modules/study/routes.ts — registra uma sessão de estudo (tela Progresso)

```
const id = insert(
  db,
  'insert into study_sessions (user_id, subject_id, started_at, day, minutes) values
(?, ?, ?, ?, ?)',
  userId, input.subjectId, input.startedAt, input.day, input.minutes,
);
```

5.3 SELECT (read)

modules/subjects/repo.ts — disciplina com contagem de tópicos, só do dono

```
export const SUBJECT_SQL = `
  select s.*,
    (select count(*) from topics t where t.subject_id = s.id) as topic_count,
    (select count(*) from topics t where t.subject_id = s.id and t.status = 'concluido') as
done_count
  from subjects s`;

export const getSubject = (db: Db, userId: number, id: number) =>
  one<SubjectRow>(db, `${SUBJECT_SQL} where s.id = ? and s.user_id = ?`, id, userId);
```

modules/goals/routes.ts — metas de um dia

```
const rows = all<GoalRow>(
  db,
  `${GOAL_SQL} where g.user_id = ? and g.goal_date = ? order by g.done, g.id`,
  req.auth!.user.id, date,
);
```

modules/access/repo.ts — log de acessos com filtros e paginação

```
export function listAccessLogs(
  db: Db,
  filter: AccessLogFilter,
  limit: number,
  offset: number,
): { items: AccessLogEntry[]; total: number } {
  const { sql, params } = whereOf(filter);
  const total = one<{ c: number }>(db, `select count(*) c from access_logs $
{sql}`, ...params)!.c;
  const rows = all<AccessLogRow>(
    db,
    `select * from access_logs ${sql} order by login_at desc, id desc limit ? offset ?`,
    ...params, limit, offset,
  );
  return { items: rows.map(toAccessLog), total };
}
```

5.4 UPDATE

modules/topics/routes.ts — muda o status e mantém done_at coerente

```
const status = patch.status ?? current.status;
// done_at coerente: preenche ao concluir, mantém se já estava concluído, limpa ao reabrir.
const doneAt = status === 'concluido' ? (current.status === 'concluido' ? current.done_at :
ctx.now()) : null;
run(
  db,
  'update topics set title = ?, notes = ?, due_date = ?, status = ?, done_at = ? where id = ?',
  patch.title ?? current.title,
  patch.notes === undefined ? current.notes : patch.notes || null,
```

```
  patch.dueDate === undefined ? current.due_date : patch.dueDate,
  status,
  doneAt,
  current.id,
);
```

modules/auth/routes.ts — atualiza o perfil (tela Perfil)

```
run(
  db,
  `update users set name = coalesce(?, name), level = coalesce(?, level),
     weekly_goal_minutes = coalesce(?, weekly_goal_minutes) where id = ?`,
  patch.name ?? null, patch.level ?? null, patch.weeklyGoalMinutes ?? null, id,
);
```

5.5 DELETE

modules/subjects/routes.ts — exclui a disciplina (apaga em cascata)

```
// Tópicos, metas, aulas, trabalhos e conversas ligados caem em cascata.
r.delete('/subjects/:id', (req, res) => {
  const s = requireSubject(db, req.auth!.user.id, intParam(req));
  run(db, 'delete from subjects where id = ?', s.id);
  res.status(204).end();
});
```

modules/admin/routes.ts — exclui usuário (transação)

```
access.endAllForUser(target.id, 'admin'); // fecha os logs abertos antes de apagar a conta
run(db, 'delete from users where id = ?', target.id);
```

5.6 Transações e importação

modules/access/service.ts — login: sessão + log + último acesso, tudo ou nada

```
const sessionId = db.transaction(() => {
  const id = insertSession(db, {
    userId: user.id,
    tokenHash: hashToken(token),
    device,
    userAgent: meta.userAgent?.slice(0, 300) ?? null,
    ip,
    createdAt: iso,
    lastSeenAt: iso,
    expiresAt: new Date(now.getTime() + config.sessionDays * DAY).toISOString(),
  });
  insertAccessLog(db, {
    userId: user.id,
    userName: user.name,
    userEmail: user.email,
    sessionId: id,
    method,
    device,
    ip,
    loginAt: iso,
  });
  run(db, 'update users set last_login_at = ? where id = ?', iso, user.id);
  return id;
})();
```

modules/access/service.ts — logout: encerra a sessão e fecha o log

```
function endSession(sessionId: number, reason: LogoutReason, at: string = nowIso()): boolean {
  return db.transaction(() => {
    const r = run(
      db,
      'update sessions set revoked_at = ?, revoke_reason = ? where id = ? and revoked_at is
null',
      at, reason, sessionId,
    );
    run(
      db,
      'update access_logs set logout_at = ?, logout_reason = ? where session_id = ? and
logout_at is null',
```

```
      at, reason, sessionId,
    );
    return r.changes > 0;
  })();
}
```

modules/sync/import.ts — importação idempotente de aulas (insere ou atualiza)

```
for (const l of course.lessons) {
  stats.lessons++;
  const cur = one<{ id: number; subject_id: number; date: string; start_time: string; end_time:
string; topic: string | null; room: string | null }>(
    db, 'select * from lessons where user_id = ? and external_id = ?', userId, l.externalId,
  );
  if (!cur) {
    insert(
      db,
      `insert into lessons (user_id, subject_id, external_id, date, start_time, end_time,
topic, room, updated_at)
       values (?, ?, ?, ?, ?, ?, ?, ?, ?)`,
      userId, subjectId, l.externalId, l.date, l.start, l.end, l.topic, l.room, nowIso,
    );
    stats.created++;
  } else if (
    cur.subject_id !== subjectId || cur.date !== l.date || cur.start_time !== l.start ||
    cur.end_time !== l.end || cur.topic !== l.topic || cur.room !== l.room
  ) {
    run(
      db,
      `update lessons set subject_id = ?, date = ?, start_time = ?, end_time = ?, topic = ?,
room = ?, updated_at = ?
       where id = ?`,
      subjectId, l.date, l.start, l.end, l.topic, l.room, nowIso, cur.id,
    );
    stats.updated++;
  }
}
```

6. Observação

As telas e os trechos refletem o estado do projeto em 30/10/2026. O model continua sendo desenvolvido; os trechos de código deste documento são gerados a partir dos arquivos da API a cada execução do script de build.